



Manual de práctica

Desarrollo de APIs REST

M5. Programación con PHP



Introducción

En el desarrollo actual de software, las APIs REST juegan un papel indispensable. Vivimos en un mundo interconectado, donde distintas aplicaciones y servicios necesitan comunicarse constantemente para intercambiar información de manera rápida, segura y eficiente. Las APIs REST (Representational State Transfer) no solo conectan aplicaciones, sino que también son fundamentales para el desarrollo de servicios móviles, integración con redes sociales, sistemas de pagos electrónicos y hasta plataformas IoT (Internet de las Cosas). Empresas líderes como Google, Facebook, Twitter y Amazon dependen totalmente de APIs REST, para brindar servicios robustos y escalables a millones de usuarios en todo el mundo.

Para aprender a desarrollar APIs REST, trabajaremos con Laravel, uno de los frameworks más populares y robustos de PHP. Laravel simplifica el proceso de creación de APIs, mediante características como Eloquent, que es un Object-Relational Mapping (ORM) poderoso, por medio del cual, se facilita la interacción con bases de datos. Así, podemos desarrollar sistemas de autenticación seguros, con un manejo eficiente de rutas, y una estructura clara, basada en el patrón Modelo-Vista-Controlador (MVC).

Recursos necesarios



Equipo

- Windows 10 u 11, x86_64, 22H2 o superior.
- 8GB de RAM
- WSL 2
- Virtualización habilitada en BIOS

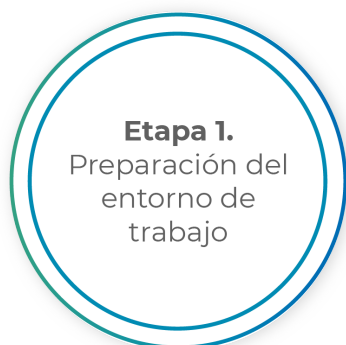


Software, sitios web, etc.

- Git (<https://git-scm.com/downloads>)
- Docker Desktop (<https://www.docker.com/products/docker-desktop/>)
- Visual Studio Code (<https://code.visualstudio.com/download>)
- Postman (<https://www.postman.com/downloads/>)

Etapas

Antes de iniciar, es necesario que identifiques las etapas con las que cuenta esta práctica.



Desarrollo de la práctica



Notas

Las capturas de pantalla que verás en este manual corresponden a un equipo con **sistema operativo Windows**, sin embargo, todos los comandos funcionan igual en cualquier sistema operativo por lo que **esta práctica funciona correctamente tanto en Windows como Linux y Mac OS**.

Etapa 1. Preparación del entorno de trabajo

Antes de comenzar, para facilitar aún más el proceso, utilizaremos una imagen Docker proporcionada por Bitnami, que contiene un entorno Laravel ya configurado, incluyendo todos los componentes necesarios como PHP, Composer, bases de datos y servidor web. Esto nos permitirá enfocarnos rápidamente en el desarrollo de nuestra API REST.



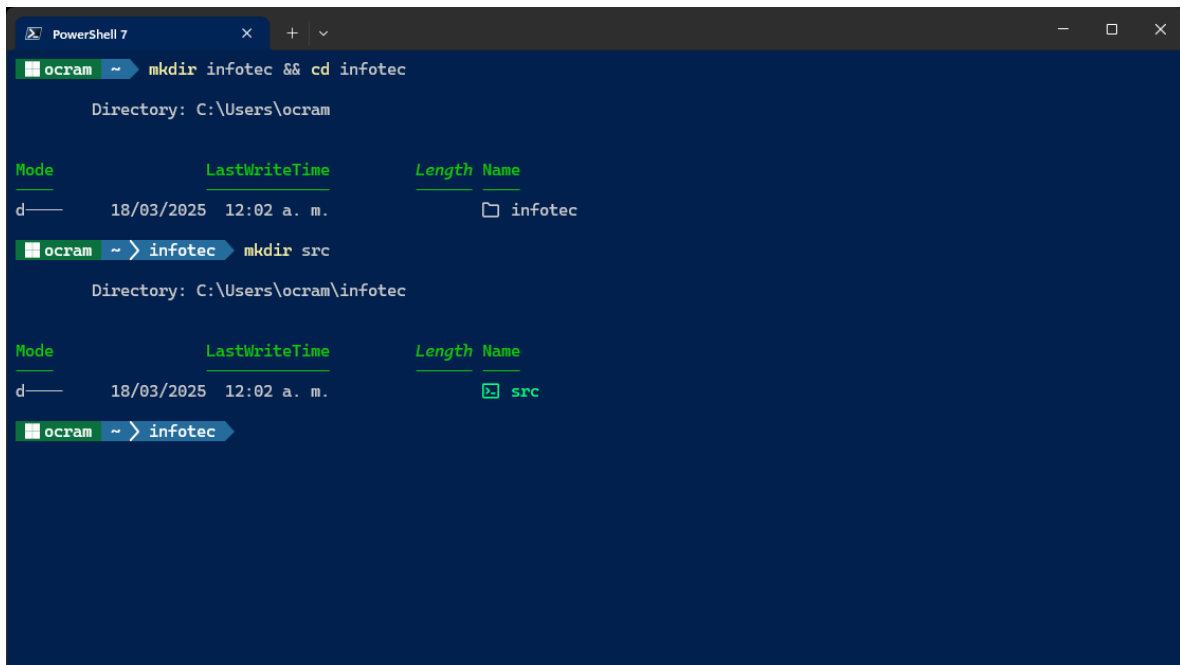
Para saber más

Bitnami es una reconocida empresa que forma parte de VMware, especializada en proporcionar paquetes y aplicaciones preconfiguradas, listas para usar, optimizadas y seguras para entornos en la nube y locales.

1. **Lanza** una terminal con el shell (intérprete de comandos) de tu preferencia, en este caso, utilizamos PowerShell.
2. **Crea** un par de carpetas para preparar el *directorio* de trabajo, ejecutando los comandos:

a. `mkdir infotec && cd infotec`

b. `mkdir src`



```
PowerShell 7
ocram ~ mkdir infotec && cd infotec

Directory: C:\Users\ocram

Mode                LastWriteTime         Length Name
----                -
d-----         18/03/2025 12:02 a. m.             infotec

ocram ~ > infotec mkdir src

Directory: C:\Users\ocram\infotec

Mode                LastWriteTime         Length Name
----                -
d-----         18/03/2025 12:02 a. m.             src

ocram ~ > infotec
```



Notas

Puedes crear las carpetas en [cualquier ruta de tu computadora](#) en la que desees preparar el directorio de trabajo, sólo recuerda ajustar los comandos necesarios para considerar el cambio.

- 3. Lanza** una instancia del editor de código Visual Studio Code, especificando la ruta recién generada como directorio de trabajo, utilizando el siguiente comando:

a. `code .`



Notas

Visual Studio Code te pedirá confirmar que confías en el autor del directorio de trabajo que estás abriendo, ya que ha detectado que es un repositorio nuevo.

Debes presionar “*Sí, confío en el autor*”.

- 4.** Crea un archivo llamado `docker-compose.yml` utilizando la opción *Archivo -> Nuevo* del menú.

5. En el contenido del archivo, **escribe** el siguiente código:

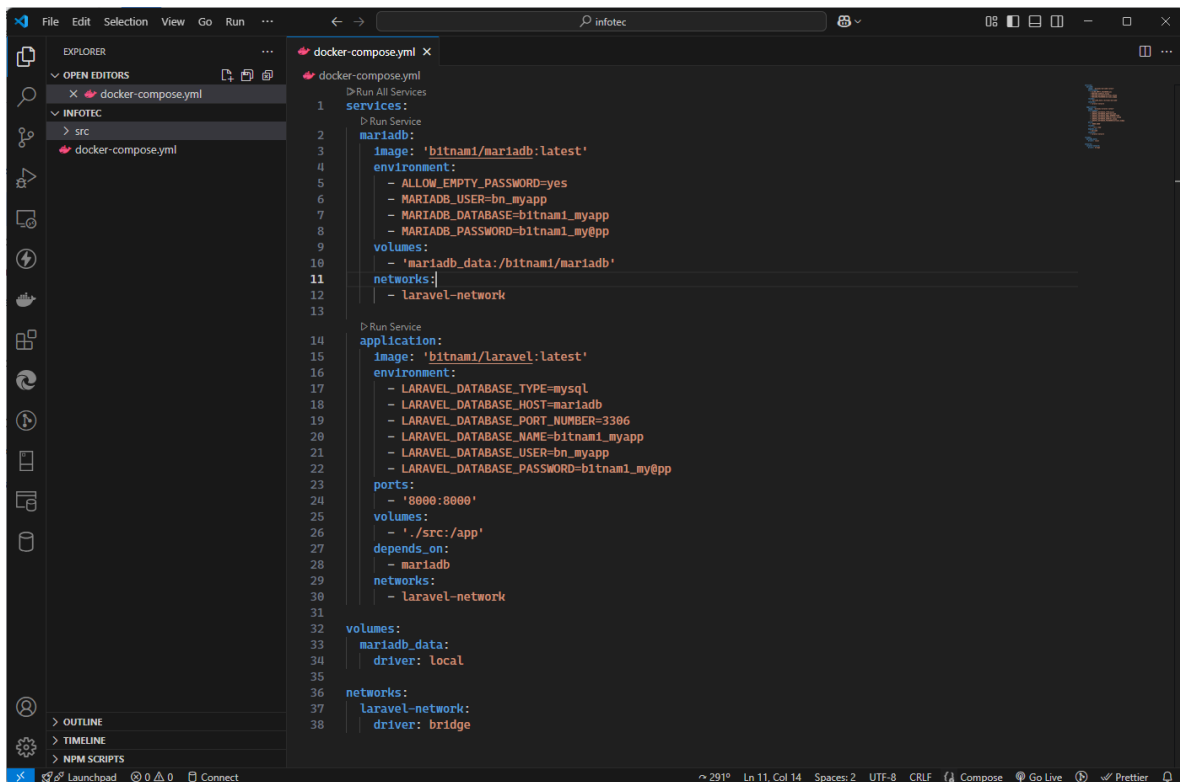
`docker-compose.yml`

```
services:
  mariadb:
    image: 'bitnami/mariadb:latest'
    environment:
      - ALLOW_EMPTY_PASSWORD=yes
      - MARIADB_USER=bn_myapp
      - MARIADB_DATABASE=bitnami_myapp
      - MARIADB_PASSWORD=b1tnam1_my@pp
    volumes:
      - 'mariadb_data:/bitnami/mariadb'
    networks:
      - laravel-network

  application:
    image: 'bitnami/laravel:latest'
    environment:
      - LARAVEL_DATABASE_TYPE=mysql
      - LARAVEL_DATABASE_HOST=mariadb
      - LARAVEL_DATABASE_PORT_NUMBER=3306
      - LARAVEL_DATABASE_NAME=bitnami_myapp
      - LARAVEL_DATABASE_USER=bn_myapp
      - LARAVEL_DATABASE_PASSWORD=b1tnam1_my@pp
    ports:
      - '8000:8000'
    volumes:
      - './src:/app'
    depends_on:
      - mariadb
    networks:
      - laravel-network

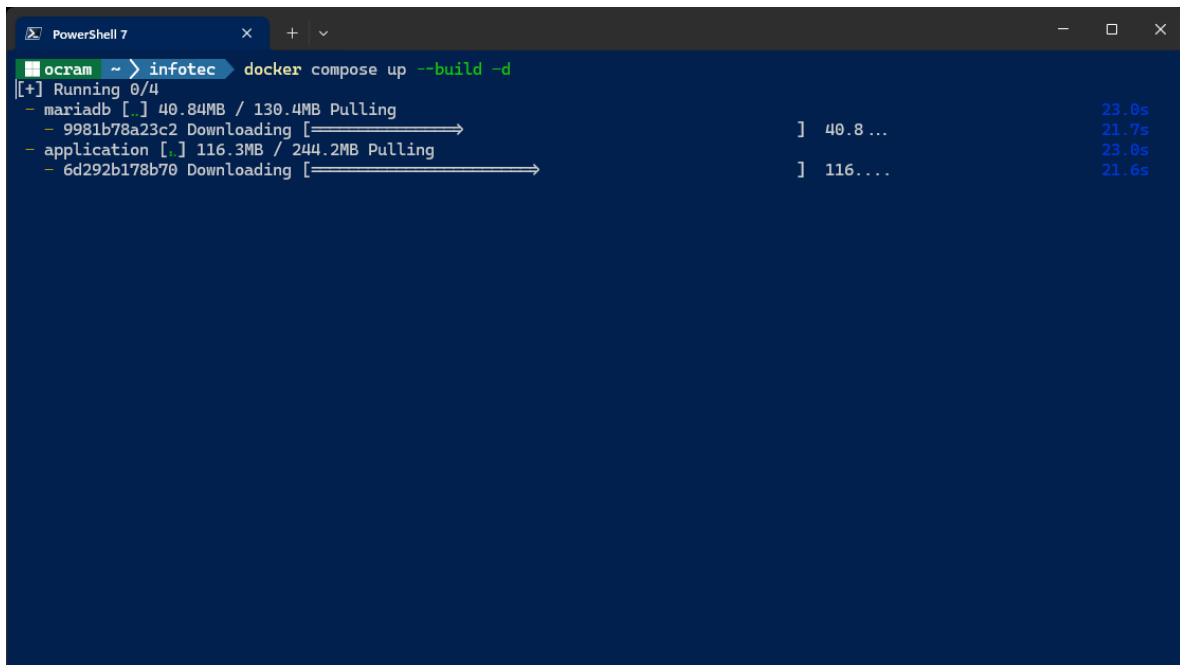
volumes:
  mariadb_data:
    driver: local

networks:
  laravel-network:
    driver: bridge
```



6. Ahora que tenemos preparado nuestro directorio de trabajo y el archivo que le especifica a Docker los servicios que deberá proporcionarnos, debemos lanzar los contenedores que albergarán nuestro entorno de trabajo. Para ello, **ejecuta** el comando:

a. `docker compose up --build -d`



```
PowerShell 7
ocram ~ > infotec docker compose up --build -d
[+] Running 0/4
- mariadb [.] 40.84MB / 130.4MB Pulling 23.0s
- 9981b78a23c2 Downloading [====>] 21.7s
- application [.] 116.3MB / 244.2MB Pulling 23.0s
- 6d292b178b70 Downloading [====>] 21.6s
```



Notas

Asegúrate de que Docker Engine esté corriendo antes de intentar lanzar los contenedores o la terminal te mostrará un mensaje de **error** que inicia con: "**unable to get image..**". Para solucionarlo, basta con iniciar Docker Engine en tu equipo e intentarlo de nuevo.

Lo que pasa en este momento es que Docker está leyendo las instrucciones del archivo YAML (Creado y llamado `docker-compose.yml` en el paso 4), descargando las imágenes necesarias y ejecutando comandos preestablecidos en la imagen. Este proceso puede tardar, dependiendo de la velocidad de tu conexión a Internet y los recursos disponibles de tu equipo. Sin embargo, sólo ocurrirá la primera vez, ya que Docker almacena en caché las imágenes y los artefactos compilados, lo que hará que el proceso sea mucho más rápido en ejecuciones posteriores. **Debes esperar a que la terminal indique que todos los servicios están en funcionamiento**, como se muestra en la siguiente imagen:

```
PowerShell 7
ocram ~ > infotec docker compose up --build -d
[+] Running 4/4
  ✓ mariadb Pulled 51.7s
  ✓ 9981b78a23c2 Pull complete 50.3s
  ✓ application Pulled 47.3s
  ✓ 6d292b178b70 Pull complete 45.8s
[+] Running 4/4
  ✓ Network infotec_laravel-network Created 0.0s
  ✓ Volume "infotec_mariadb_data" Created 0.0s
  ✓ Container infotec-mariadb-1 Started 1.6s
  ✓ Container infotec-application-1 Started 0.4s
ocram ~ > infotec
```



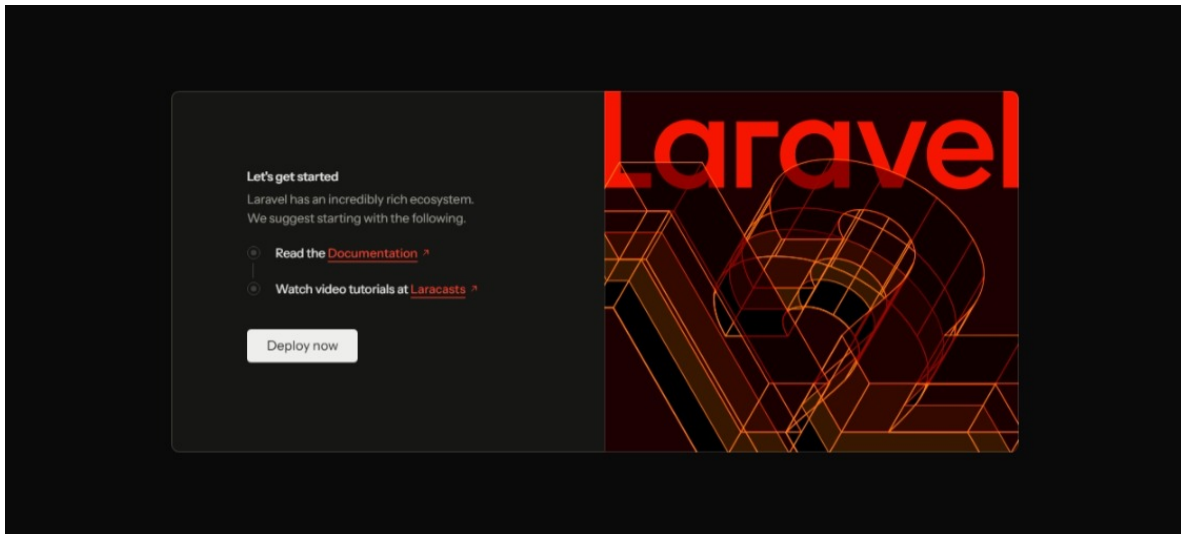
Para saber más

Durante la ejecución de los contenedores, las aplicaciones registran mensajes en una **bitácora** con información relevante, como los **errores**, **mensajes de advertencia**, etc. Para revisarlas de manera rápida y sencilla, puedes instalar *Docker Desktop* para tu sistema operativo.

Docker Desktop es una aplicación multiplataforma que permite ejecutar y gestionar fácilmente contenedores Docker en Windows, Mac y Linux, ofreciendo una interfaz gráfica para simplificar todas las operaciones relacionadas con contenedores, como creación, ejecución y administración.

Al finalizar la ejecución de las rutinas de inicialización de los contenedores (pasos del 1 al 6), podrás observar que la carpeta que creaste llamada **src** (comando b del paso 2) ahora contiene una gran cantidad de directorios y archivos. Estos pertenecen a tu proyecto en Laravel.

7. Verifica que el proyecto está corriendo: **Accede** a la URL <http://localhost:8000> en el navegador de tu preferencia. Deberás ver una página inicial como en la siguiente imagen:



¡Felicidades! Ya cuentas con un entorno de trabajo completamente configurado y listo para comenzar. Has concluido con éxito la primera etapa de este manual de práctica.



- Recuerda: siempre que quieras acceder a la base de datos, el contenedor debe estar **activo**.
- Puedes **detener** y **eliminar** los contenedores desde Docker Desktop o desde la terminal, utilizando el comando **docker compose down**.

Etapa 2. Creando tu primera API REST

El objetivo de esta etapa es poner en práctica tus conocimientos previos para planear, diseñar y desarrollar una API REST funcional. Aprenderás a definir claramente los endpoints de tu aplicación, implementar controladores API que gestionen solicitudes HTTP y manejar datos en formato JSON. Finalmente, utilizaremos Postman, una herramienta poderosa para probar y verificar el correcto funcionamiento de la API que desarrollarás.

Para entrar en contexto, desarrollaremos **una API para una empresa dedicada a la organización de conferencias y eventos**. La API deberá gestionar:

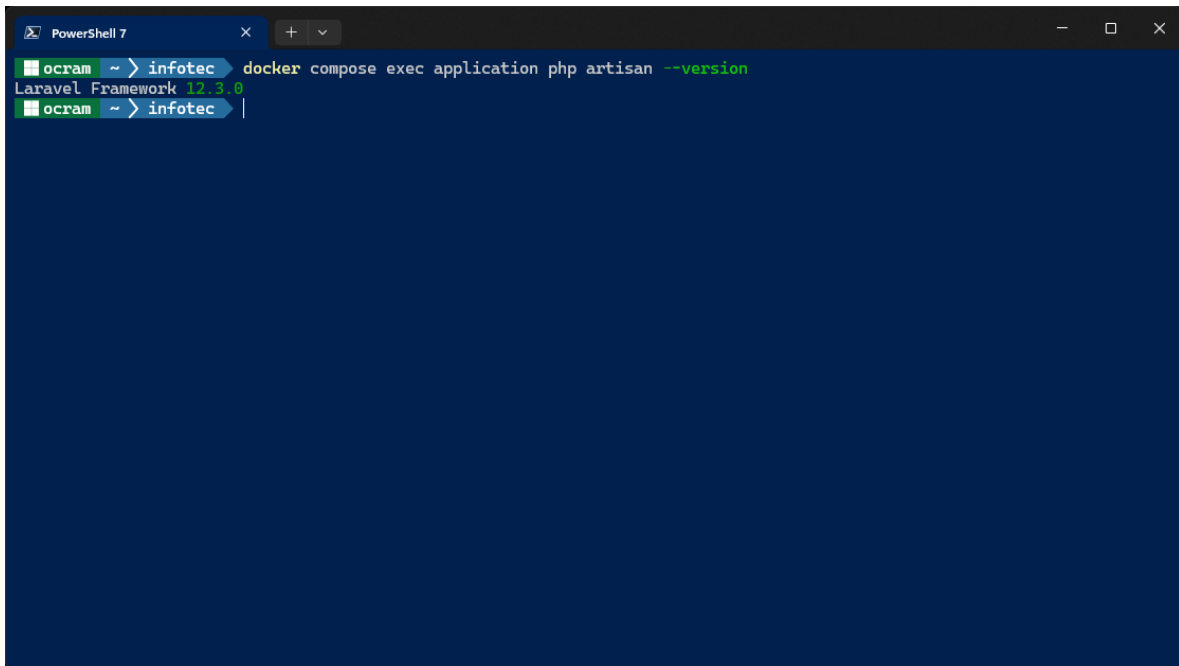
- Eventos
- Ponentes
- Asistentes

Siguiendo la arquitectura REST, cada uno de estos 3 controladores debe contener operaciones CRUD en los métodos:

- GET; para obtener todos los elementos.
- GET; para obtener un elemento por su identificador.
- POST; para crear nuevos elementos.
- PUT; para actualizar un elemento.
- DELETE; para eliminar un elemento.

1. Primero, debemos verificar si podemos ejecutar comandos *artisan*. Para hacerlo, regresa a la terminal y **ejecuta** el comando:

a. `docker compose exec application php artisan --version`



```
PowerShell 7
ocram ~ > infotec docker compose exec application php artisan --version
Laravel Framework 12.3.0
ocram ~ > infotec |
```



Notas

Recuerda que la versión de los comandos puede variar, si Bitnami ha actualizado su imagen a la más reciente de Laravel.

2. Por default, la imagen Laravel de Bitnami utiliza el motor de base de datos de *SQLite*, pero en nuestro caso, debemos utilizar el servicio *MariaDB*, el cual lanzaremos en nuestro archivo `docker-compose.yml`. Para indicarle a Laravel que utilice nuestro motor de

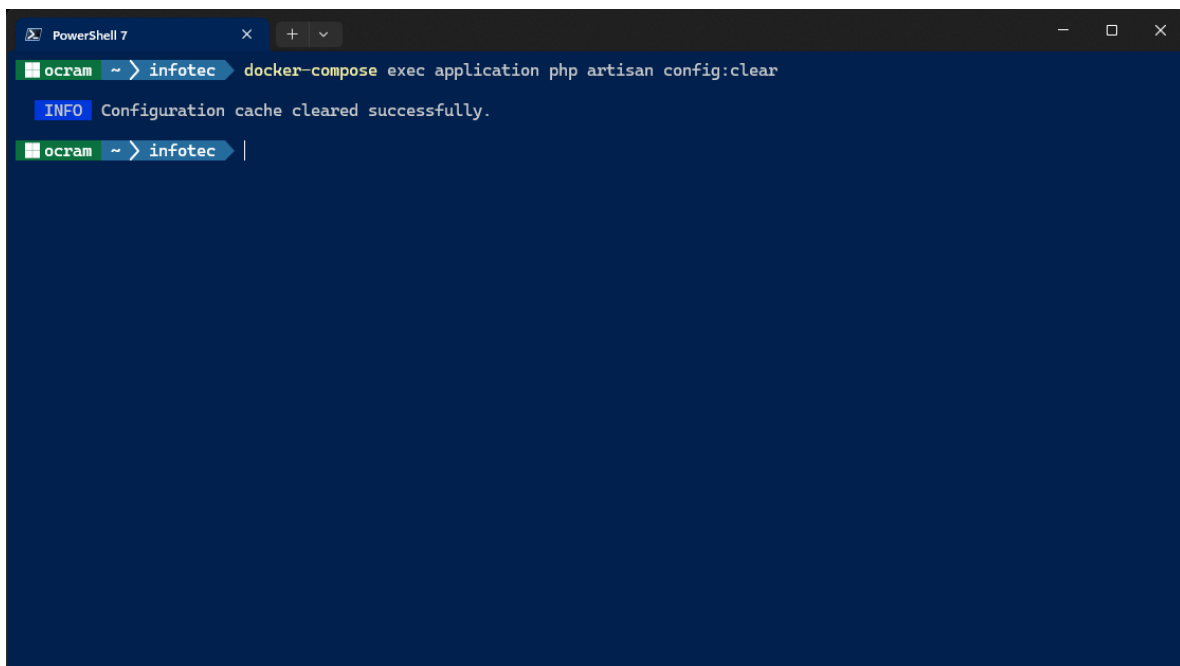
bases de datos, **ubica** el archivo **.env** y **edita** las siguientes variables, para que queden de la siguiente manera:

.env

```
DB_CONNECTION=mysql
DB_HOST=mariadb
DB_PORT=3306
DB_DATABASE=bitnami_myapp
DB_USERNAME=bn_myapp
DB_PASSWORD=b1tnam1_my@pp
```

3. Para asegurar que los cambios del paso anterior tengan efecto, debemos notificar a Laravel que recargue la configuración. Por lo cual, **ejecuta** el siguiente comando:

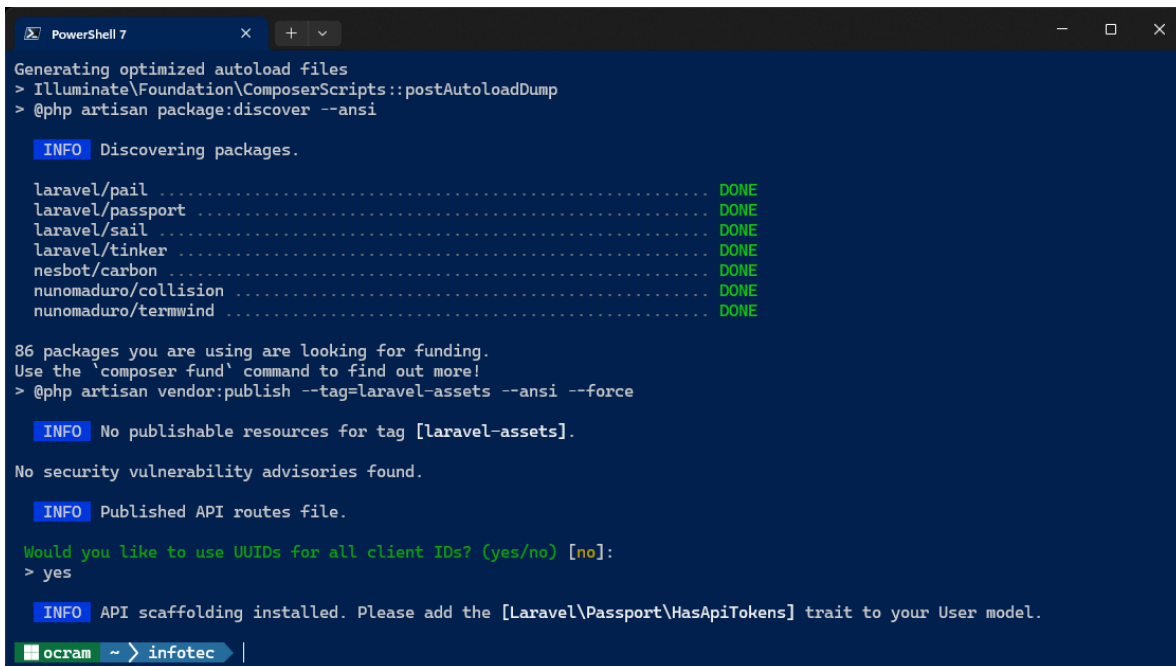
a. **docker-compose exec application php artisan config:clear**



```
PowerShell 7
ocram ~ > infotec docker-compose exec application php artisan config:clear
INFO Configuration cache cleared successfully.
ocram ~ > infotec |
```

4. El proyecto inicial de Laravel no está configurado para utilizar enrutamiento de APIs, así que debes agregarlo manualmente. Para ello, **ejecuta** en la terminal el comando:

a. `docker compose exec application php artisan install:api --passport`



```
PowerShell 7
Generating optimized autoload files
> Illuminate\Foundation\ComposerScripts::postAutoloadDump
> @php artisan package:discover --ansi

INFO Discovering packages.

laravel/pail ..... DONE
laravel/passport ..... DONE
laravel/sail ..... DONE
laravel/tinker ..... DONE
nesbot/carbon ..... DONE
nunomaduro/collision ..... DONE
nunomaduro/termwind ..... DONE

86 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

INFO No publishable resources for tag [laravel-assets].

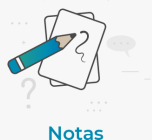
No security vulnerability advisories found.

INFO Published API routes file.

Would you like to use UUIDs for all client IDs? (yes/no) [no]:
> yes

INFO API scaffolding installed. Please add the [Laravel\Passport\HasApiTokens] trait to your User model.

ocram ~ > infotec |
```

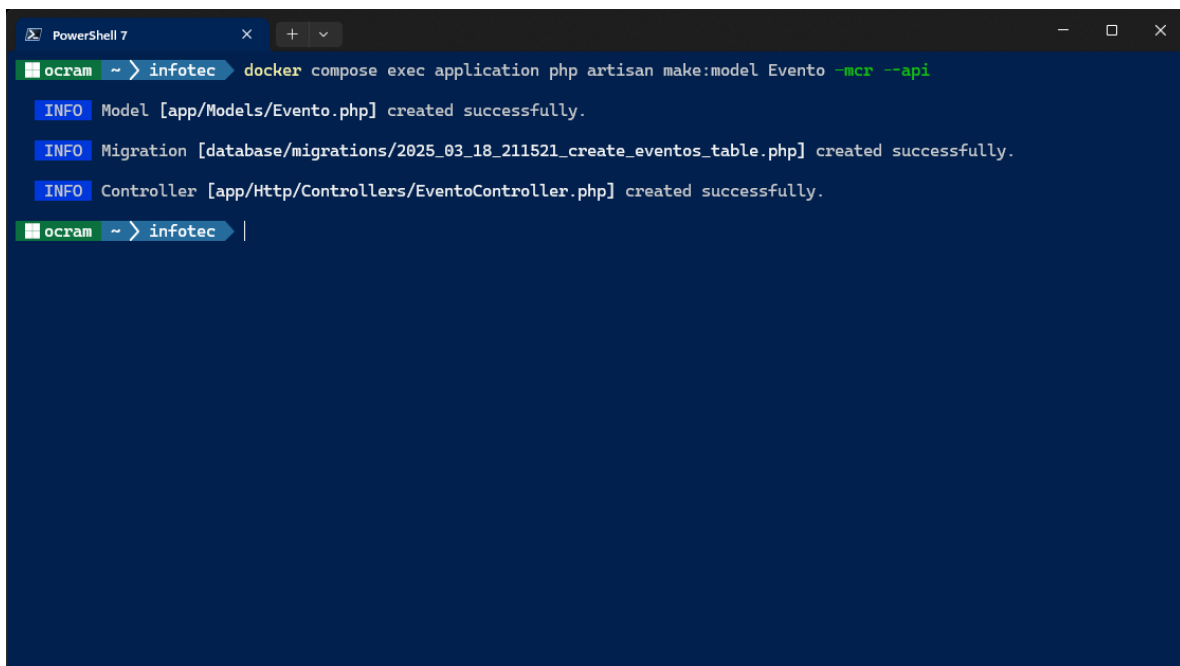


Notas

Durante el proceso de instalación del enrutador de APIs, el comando te preguntará si deseas utilizar UUIDs (Identificador Universalmente Único) como identificadores de los clientes, debes escribir **yes** (como se muestra en la imagen anterior) y presionar la tecla **enter**.

5. Una vez que has configurado Laravel para utilizar controladores APIs, debes crear un controlador para gestionar todas las peticiones relacionadas a la gestión de eventos. Con este propósito, **ejecuta** el siguiente comando:

a. `docker compose exec application php artisan make:model Evento -mcr --api`



```
PowerShell 7
ocram ~ > infotec docker compose exec application php artisan make:model Evento -mcr --api
INFO Model [app/Models/Evento.php] created successfully.
INFO Migration [database/migrations/2025_03_18_211521_create_eventos_table.php] created successfully.
INFO Controller [app/Http/Controllers/EventoController.php] created successfully.
ocram ~ > infotec |
```

Este comando crea los archivos listados en la terminal. El archivo `Evento.php` contendrá el modelo de los datos relacionados con la entidad *Evento*, el archivo `EventoController.php` contendrá la lógica de negocio relacionada con la misma entidad y el archivo `AAAA_MM_DD_hhmmss_create_eventos_table.php` contiene la definición de la estructura de la tabla correspondiente a la misma entidad en la base de datos.

6. **Ubica** el archivo `AAAA_MM_DD_hhmmss_create_eventos_table.php` (AAAA_MM_DD_hhmmss hace referencia al día y la hora en la que se generó) en el directorio `database\migrations` y **edítalo** de la siguiente manera:

AAAA_MM_DD_hhmmss_create_eventos_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('eventos', function (Blueprint $table) {
            $table->id();
            $table->string('titulo');
            $table->string('descripcion');
            $table->date('fecha_inicio');
            $table->date('fecha_fin');
            $table->string('ubicacion');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('eventos');
    }
};
```

Si observas detenidamente el método `up()`, te darás cuenta que se han agregado 5 campos más para datos que queremos almacenar acerca de los eventos.

7. Para reflejar estos nuevos campos en el modelo de la entidad *Evento*, **busca** el directorio `app\Models` y **edita** el archivo llamado `Evento.php` para que quede de la siguiente manera:

Evento.php

```
<?php

namespace App\Models;

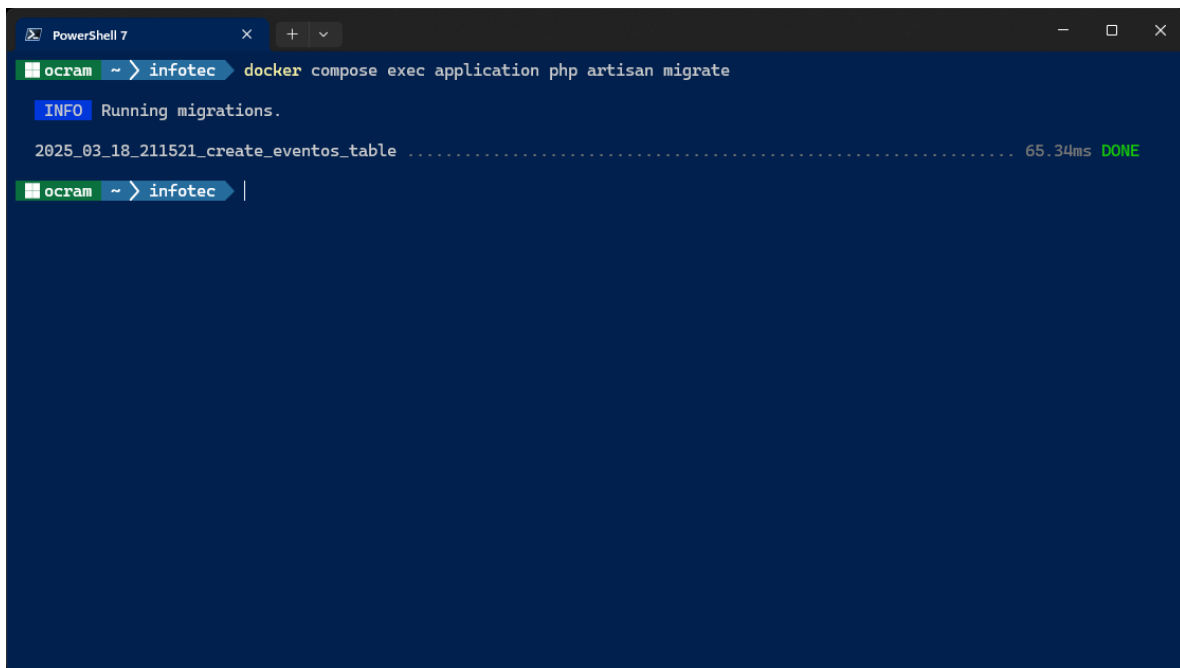
use Illuminate\Database\Eloquent\Model;

class Evento extends Model
{
    protected $table = 'eventos';
    protected $fillable = [
        'titulo',
        'descripcion',
        'fecha_inicio',
        'fecha_fin',
        'ubicacion'
    ];
}
```

Presta atención a la variable `$table` en ella debes especificar el nombre de la tabla que estableciste en el método `up()` de la **migración** (serie de instrucciones que deben ejecutarse en una base de datos para aplicar los cambios necesarios por la programación) que creaste en el paso anterior.

8. Ahora que tenemos definida la estructura de la entidad *Evento*, debemos aplicar la **migración** (los cambios) a la base de datos. Para ello, **ejecuta** el comando:

a. `docker compose exec application php artisan migrate`



```
PowerShell 7
ocram ~ > infotec docker compose exec application php artisan migrate
INFO Running migrations.
2025_03_18_211521_create_eventos_table ..... 65.34ms DONE
ocram ~ > infotec |
```

Al aplicar la **migración**, se generó una tabla en la base de datos con las columnas que especificaste en el modelo de la entidad *Evento*, en ella se almacenará la información relacionada a los eventos.

Ahora debes **codificar la programación** que se encargará de almacenar, recuperar y transformar la información, utilizando la lógica de negocio requerida por el aplicativo. En esta ocasión utilizaremos una lógica de negocio sencilla y sin restricciones.

9. Ubica el archivo `EventoController.php` en el directorio `app\Controllers`. Este archivo está generado en base a una plantilla genérica (todos los controladores que Laravel genera son iguales). **Inspecciona** cada uno de sus métodos.

- **index.** Se utilizará para recuperar todos los elementos de la tabla relacionada.
- **store.** Será el encargado de almacenar un nuevo elemento en la tabla.
- **show.** Se utilizará para recuperar un elemento en específico.
- **update.** Actualizará un elemento en específico.
- **destroy.** Eliminará definitivamente un elemento en específico.

10. Edita el archivo como se muestra a continuación:

a. Analiza el código prestando atención a los comentarios, para que puedas comprender qué está sucediendo.

`EventoController.php`

```
<?php

namespace App\Http\Controllers;

// Agregar el modelo Evento
use App\Models\Evento;
// Agregar la clase Request para manejar las peticiones
use Illuminate\Http\Request;
// Agregar la clase Validator para validar los datos de la petición
use Illuminate\Support\Facades\Validator;
```

```
class EventoController extends Controller
{
    /**
     * Mostrar una lista del recurso.
     */
    public function index()
    {
        // Recuperar todos los recursos
        $eventos = Evento::all();

        // Retornar los recursos recuperados
        $respuesta = [
            'eventos' => $eventos,
            'status' => 200,
        ];
        return response()->json($respuesta);
    }

    /**
     * Almacenar un recurso recién creado en el almacenamiento.
     */
    public function store(Request $request)
    {
        // Validar que la petición contenga todos los datos necesarios
        $validator = Validator::make($request->all(), [
            'titulo' => 'required',
            'descripcion' => 'required',
            'fecha_inicio' => 'required|date',
            'fecha_fin' => 'required|date',
            'ubicacion' => 'required',
        ]);

        // Si la petición no contiene todos los datos necesarios,
        // retornar un mensaje de error
        if ($validator->fails()) {
            $respuesta = [
                'message' => 'Datos faltantes',
                'status' => 400, // Petición inválida
            ];
        }
    }
}
```

```
        return response()->json($respuesta, 400);
    }

    // Crear un nuevo recursos con los datos de la petición
    $evento = Evento::create([
        'titulo' => $request->titulo,
        'descripcion' => $request->descripcion,
        'fecha_inicio' => $request->fecha_inicio,
        'fecha_fin' => $request->fecha_fin,
        'ubicacion' => $request->ubicacion,
    ]);

    // Si el recurso no se pudo crear, retornar un mensaje de
error
    if (!$evento) {
        $respuesta = [
            'message' => 'Error al crear el evento',
            'status' => 500, // Error interno del servidor
        ];
        return response()->json($respuesta, 500);
    }

    // Retornar el recurso creado
    $respuesta = [
        'evento' => $evento,
        'status' => 201,
    ];
    return response()->json($respuesta, 201);
}

/**
 * Mostrar el recurso especificado.
 */
public function show($id)
{
    // Recuperar el recurso especificado
    $evento = Evento::find($id);
```

```
// Si el recurso no se pudo recuperar, retornar un mensaje de
error
if (!$evento) {
    $respuesta = [
        'message' => 'Evento no encontrado',
        'status' => 404, // No encontrado
    ];
    return response()->json($respuesta, 404);
}

// Retornar el recurso recuperado
$respuesta = [
    'evento' => $evento,
    'status' => 200, // OK
];
return response()->json($respuesta);
}

/**
 * Actualizar el recurso especificado en el almacenamiento.
 */
public function update(Request $request, $id)
{
    // Recuperar el recurso especificado
    $evento = Evento::find($id);

    // Si el recurso no se pudo recuperar, retornar un mensaje de
error
    if (!$evento) {
        $respuesta = [
            'message' => 'Evento no encontrado',
            'status' => 404, // No encontrado
        ];
        return response()->json($respuesta, 404);
    }

    // Validar que la petición contenga todos los datos necesarios
    $validator = Validator::make($request->all(), [
        'titulo' => 'required',
```

```
        'descripcion' => 'required',
        'fecha_inicio' => 'required|date',
        'fecha_fin' => 'required|date',
        'ubicacion' => 'required',
    ]);

    // Si la petición no contiene todos los datos necesarios,
    retornar un mensaje de error
    if ($validator->fails()) {
        $respuesta = [
            'message' => 'Datos faltantes',
            'status' => 400, // Petición inválida
        ];
        return response()->json($respuesta, 400);
    }

    // Actualizar el recurso especificado con los datos de la
    petición
    $evento->titulo = $request->titulo;
    $evento->descripcion = $request->descripcion;
    $evento->fecha_inicio = $request->fecha_inicio;
    $evento->fecha_fin = $request->fecha_fin;
    $evento->ubicacion = $request->ubicacion;
    $evento->save();

    // Retornar el recurso actualizado
    $respuesta = [
        'evento' => $evento,
        'status' => 200, // OK
    ];
    return response()->json($respuesta);
}

/**
 * Eliminar el recurso especificado del almacenamiento.
 */
public function destroy($id)
{
    // Recuperar el recurso especificado
```



```
$evento = Evento::find($id);

// Si el recurso no se pudo recuperar, retornar un mensaje de
error
if (!$evento) {
    $respuesta = [
        'message' => 'Evento no encontrado',
        'status' => 404, // No encontrado
    ];
    return response()->json($respuesta, 404);
}

// Eliminar el recurso especificado
$evento->delete();

// Retornar un mensaje de éxito
$respuesta = [
    'message' => 'Evento eliminado',
    'status' => 200, // OK
];
return response()->json($respuesta);
}
```

11. En este momento, ya tienes todo el código y la lógica de negocio codificada en el controlador, pero aún no está disponible para ser ejecutada. Esto se logra al **configurar** las rutas o *endpoints* en el archivo `api.php` del directorio `routes`. **Edita** el archivo para que quede de la siguiente manera:

api.php

```
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
// Agregar el controlador EventoController
use App\Http\Controllers\EventoController;

/**
 * Rutas para el recurso Evento.
 */
// Recuperar todos los eventos
Route::get('/eventos', [EventoController::class, 'index']);
// Almacenar un evento nuevo
Route::post('/eventos', [EventoController::class, 'store']);
// Recuperar un evento específico
Route::get('/eventos/{id}', [EventoController::class, 'show']);
// Actualizar un evento específico
Route::put('/eventos/{evento}', [EventoController::class, 'update']);
// Eliminar un evento específico
Route::delete('/eventos/{id}', [EventoController::class, 'destroy']);
```

Has completado todos los pasos necesarios para poder gestionar la entidad *Evento* en tu código. Ahora, debes ser capaz de ejecutar pruebas para validar que el código funciona.

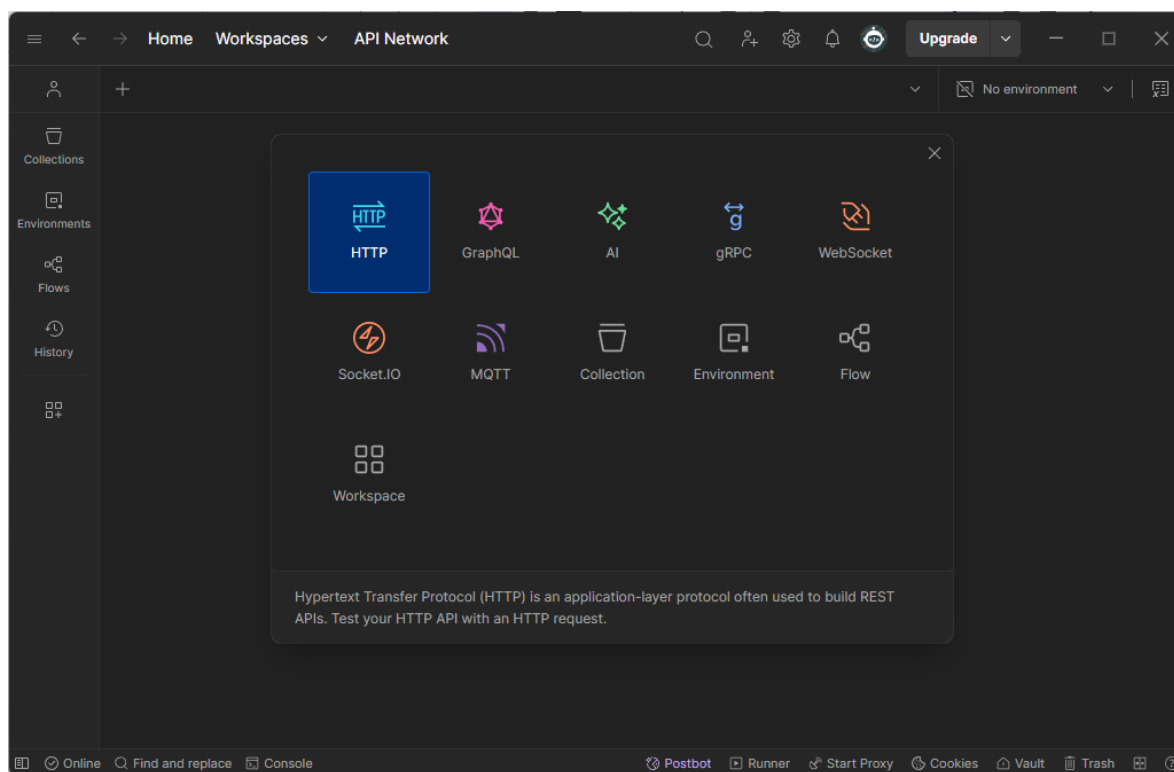
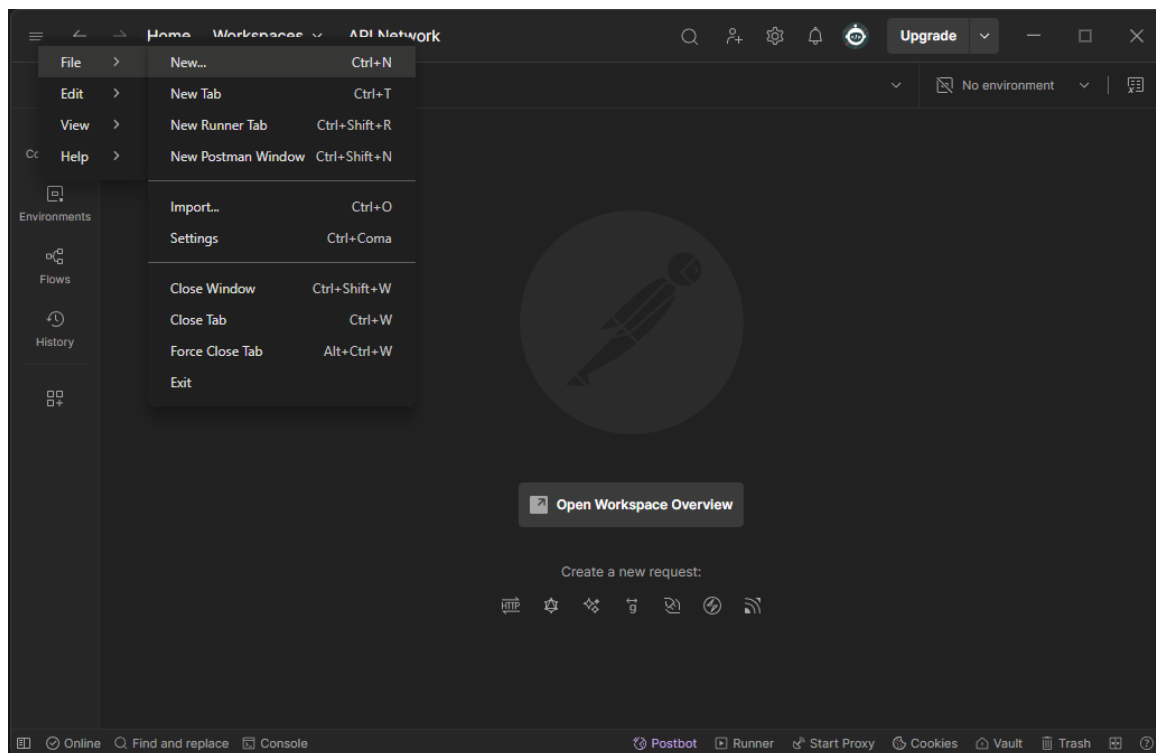
Etapa 3. Probando la API REST

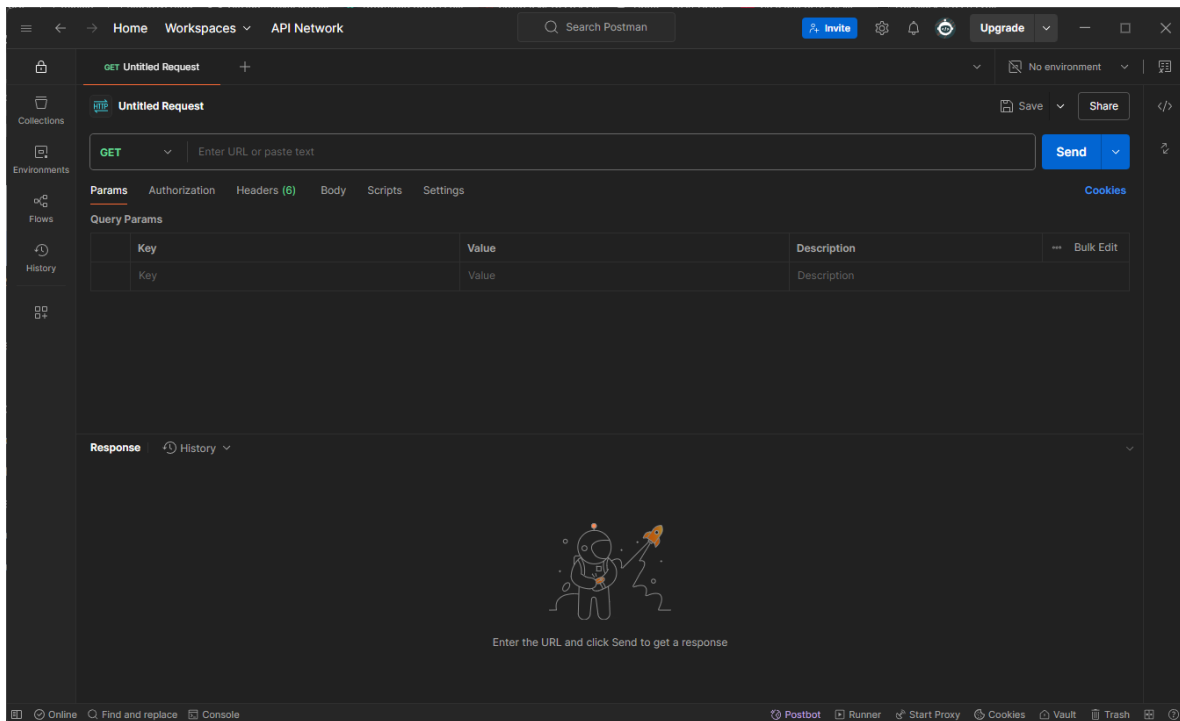
En esta etapa, utilizarás Postman, una herramienta muy útil para realizar pruebas de APIs. En ella, validarás cada uno de los métodos HTTP (GET, POST, PUT, DELETE) que configuraste en tu controlador durante la etapa anterior. A través de estas pruebas, podrás verificar que tu API REST funciona correctamente, maneja adecuadamente las solicitudes y respuestas JSON, y responde de forma precisa ante diferentes escenarios. Este paso es crucial para asegurar la calidad y robustez de tu API.

1. **Lanza** la aplicación Postman.
2. **Crea** un nuevo archivo de petición, haciendo clic en el menú *File*, luego selecciona *New*, o utiliza el atajo del teclado **Ctrl + N** y selecciona *HTTP* en la ventana emergente. Observa la secuencia de las siguientes imágenes:

Módulo 5. Programación con PHP

Manual de práctica

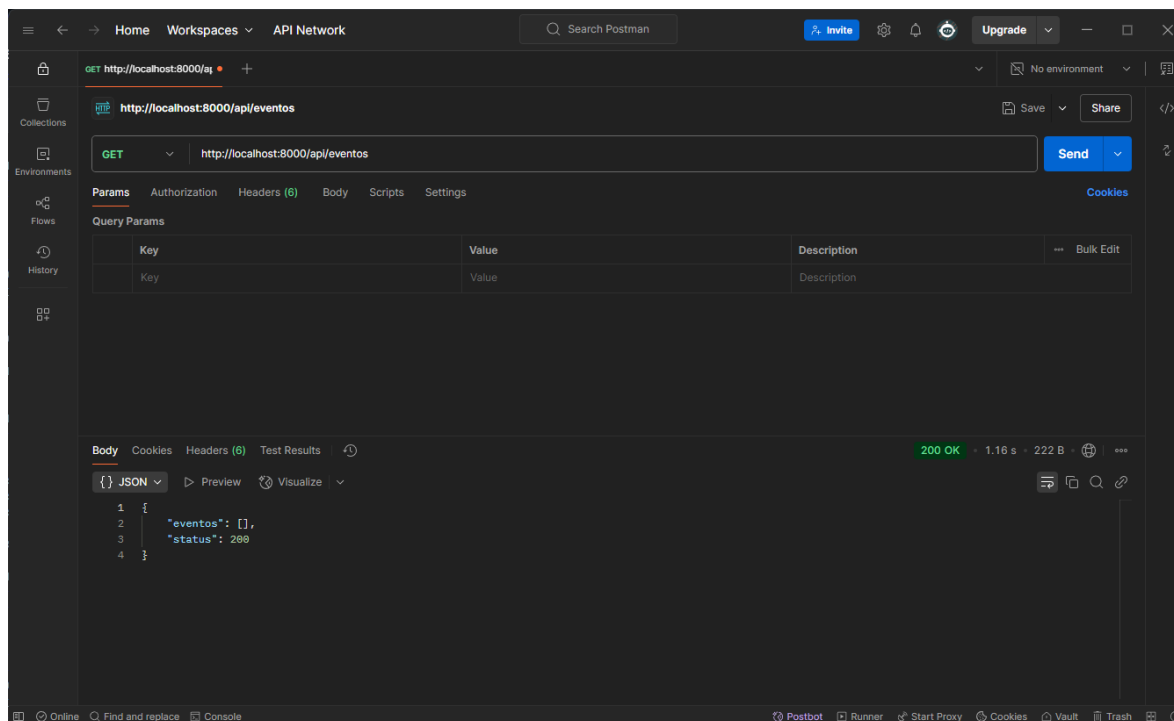




3. Iniciaremos invocando la ruta para obtener todas las entidades de eventos, que en este momento deberían estar vacías. **Selecciona** el método *GET* de la lista desplegable e **ingresa** la URL <http://localhost:8000/api/eventos> en el cuadro de texto. Deja todas las demás opciones como están y haz clic en el botón *Send*.

Módulo 5. Programación con PHP

Manual de práctica



Notarás que en la sección de abajo, se muestra la respuesta que la API envió y que consta de un objeto JSON con las propiedades **eventos**, lo cual, equivale a un **arreglo vacío**, y la propiedad *status*, indicando un **código de estado 200**.



Notas

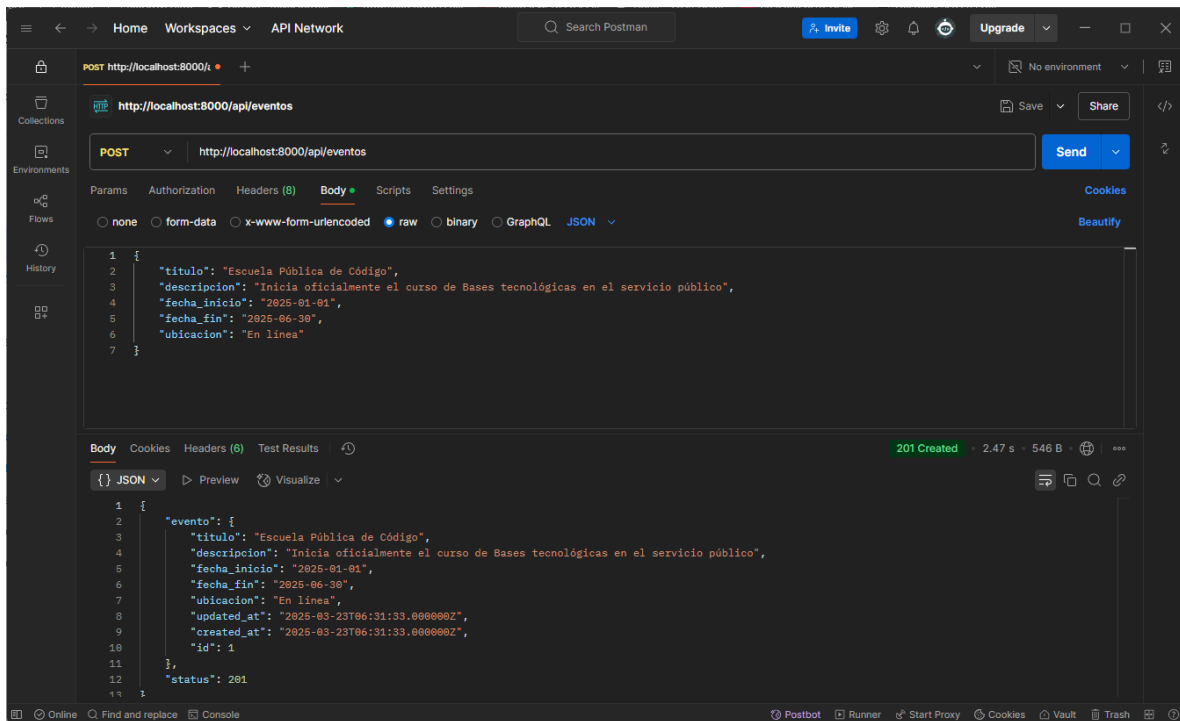
Como parte de un buen diseño de APIs REST, debemos considerar **retornar** (return) un código de estado adecuado. Entre los principales están:

- **200 Ok**. La petición fue exitosa.
- **201 Created**. Se creó correctamente el nuevo recurso.
- **202 Accepted**. Se acepta la petición, pero será procesada después.
- **204 No Content**. Petición exitosa, pero no devuelve contenido.
- **301 Moved Permanently**. El recurso solicitado cambió permanentemente de ubicación.
- **400 Bad request**. La solicitud está mal formada o contiene errores.
- **401 Unauthorized**. No autorizado, debe autenticarse.
- **403 Forbidden**. No cuenta con los permisos necesarios.
- **404 Not Found**. No se encuentra el recurso solicitado.
- **500 Internal Server Error**: Error genérico del servidor.

4. Para continuar, debes añadir un nuevo elemento a la base de datos, eso lo lograrás invocando el método *POST* de la misma ruta.

Selecciona *POST* en la lista desplegable, **ubica** la sección *Body* justo debajo del cuadro de texto con la URL, **selecciona** *raw* y **asegúrate** que JSON esté seleccionado. **Ingresa** el siguiente objeto JSON en el cuerpo de la petición y haz clic en *Send*:

```
{
  "titulo": "Escuela Pública de Código",
  "descripcion": "Inicia oficialmente el curso de Bases tecnológicas en el servicio público",
  "fecha_inicio": "2025-01-01",
  "fecha_fin": "2025-06-30",
  "ubicacion": "En línea"
}
```



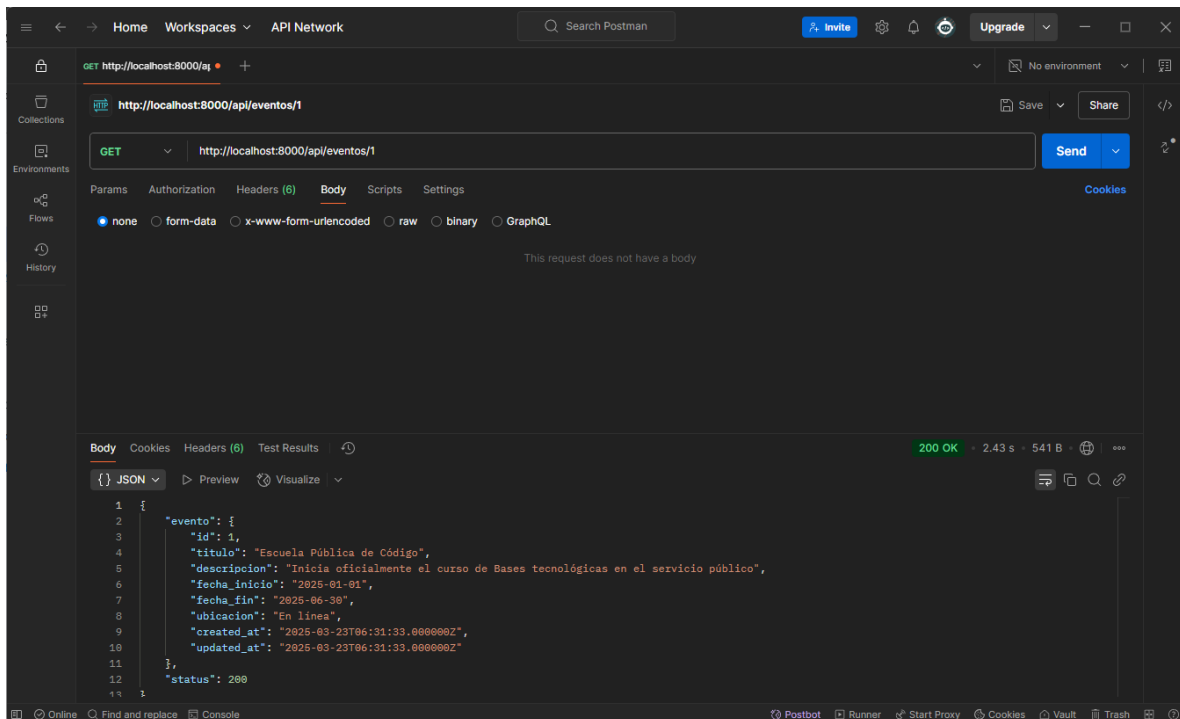
Observa cómo en la respuesta obtenida de la API, se muestra un **código de estado 201** y el cuerpo contiene la entidad recién creada. **Cambia** un poco los datos, para añadir varios eventos.



Ejercicio de reforzamiento

Invoca el listado completo de las entidades que creaste en el paso 3, para consultar las entidades del modelo Evento que has creado.

5. Ahora que cuentas con algunas entidades almacenadas en la base de datos, puedes recuperar una entidad en específico, solicitándola por su **id**. Para ello, **cambia** el método de la petición a *GET* y **agrega** el **id** de la entidad al final de la ruta, anteponiendo una barra. La URL deberá quedar de la siguiente manera <http://localhost:8000/api/eventos/{id}> y da clic en *Send*:



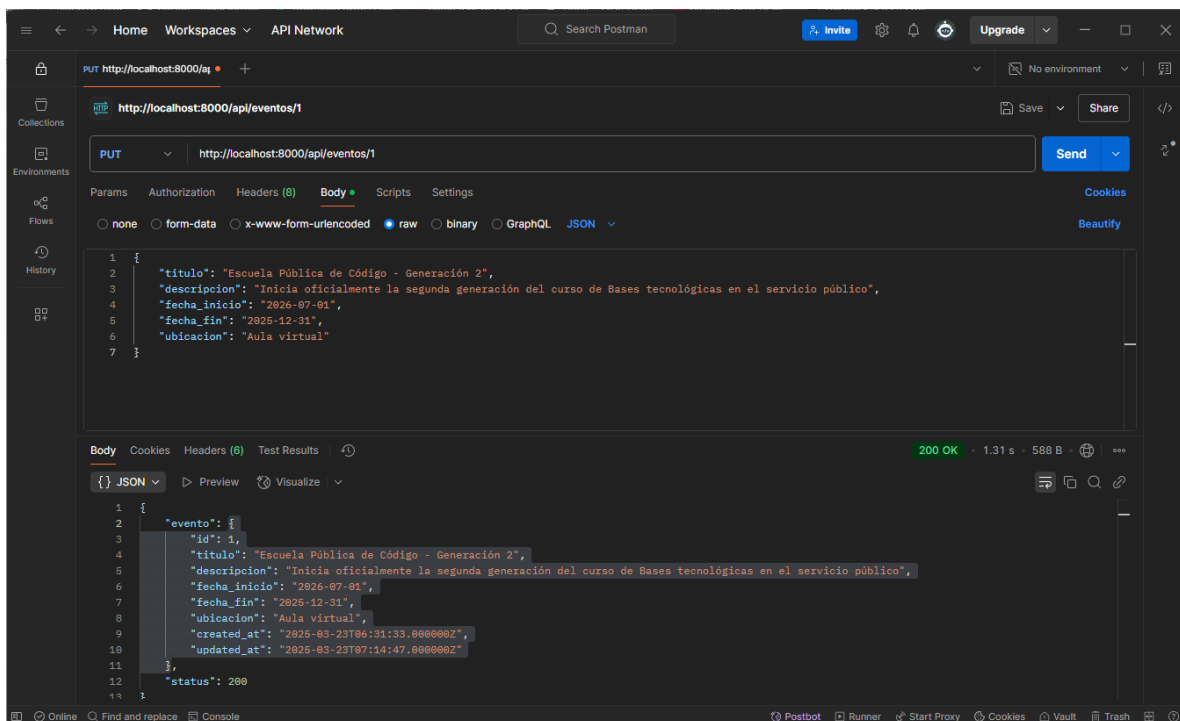
Intenta **enviar** un **id** que no existe, para que observes cómo lo maneja el servidor, retornando un **código de estado 404**.



Notas

No **elimines** el contenido del cuerpo de la petición, porque lo usaremos después. No te preocupes, al tratarse de un método *GET*, el servidor ignora el contenido del cuerpo.

6. Un proceso muy importante en cualquier software, es la actualización de entidades. Para lograrlo en tu API, **selecciona** el método *PUT* de la lista desplegable, **cambia** un poco los valores del objeto JSON que enviarás como contenido del cuerpo de la petición y **presiona** el botón *Send*.

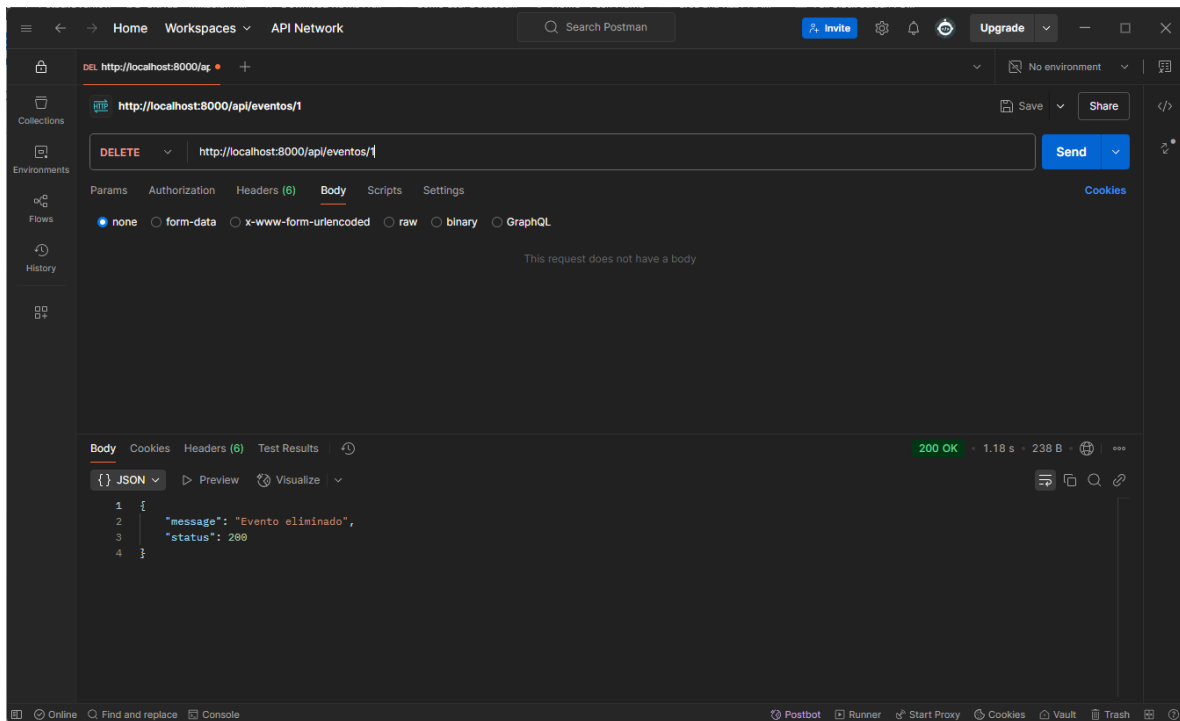


Observa que la entidad retornada por el servidor contiene los datos actualizados. Una vez más, puedes consultarla utilizando el método *GET* y enviando el `id` correspondiente como parámetro de ruta, de la misma manera que lo hiciste en el paso anterior.

7. Para eliminar una entidad, **selecciona** el método *DELETE* de la lista desplegable y **actualiza** la URL, con el propósito de enviar el `id` de la entidad que deseas eliminar.

Módulo 5. Programación con PHP

Manual de práctica



¿Crees que has entendido bien el concepto de Modelo-Vista-Controlador (MVC), enfocado a APIs REST?

Es tu turno de realizar los cambios necesarios a tu API para gestionar las entidades **Ponentes** y **Asistentes**, con los siguientes campos en cada una:



Ejercicio de
reforzamiento

- **Ponentes**
 - id
 - nombre
 - biografia
 - especialidad
- **Asistentes**
 - id
 - nombre
 - email
 - telefono
 - evento_id

¡Listo!

Has concluido exitosamente la práctica sobre Desarrollo de APIs REST.

Referencias:

- Laravel. (s.f.). *Documentación oficial*. Recuperado el 21 de marzo de 2025 de <https://laravel.com/docs>
- Mozilla Developer Network. (s.f.). *Métodos de petición HTTP*. Recuperado el 21 de marzo de 2025 de <https://developer.mozilla.org/es/docs/Web/HTTP/Methods>
- Mozilla Developer Network. (s.f.). *Códigos de estado HTTP*. Recuperado el 21 de marzo de 2025 de <https://developer.mozilla.org/es/docs/Web/HTTP/Status>